

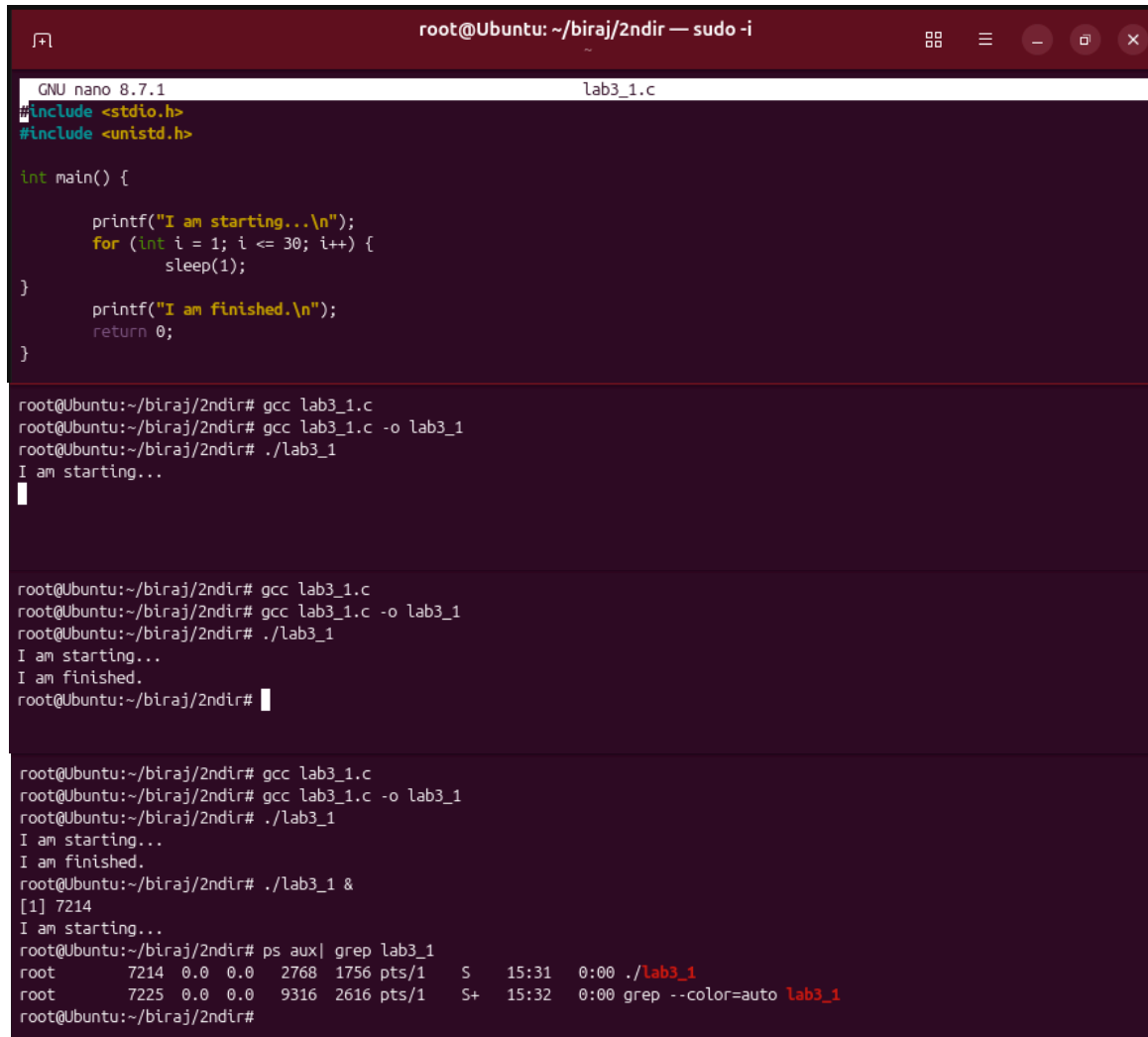
Table of Content:

Task No.1	2
Task no.2	3
Task No.3	5
Task no.4	6
Task No.5	7

Table of Figures:

Figure: Task 1 C code, commands for compilation and Execution.	3
Figure: Task 2 C code, Compilation and execution.	4
Figure: Task 3 C code, Compilation and Execution.	6
Figure: Task 4 C code, Compilation and Execution.	7
Figure: Task 5 C code, Compilation and Execution.	8

Task No.1

A terminal window titled 'root@Ubuntu: ~/biraj/2ndir — sudo -i' displays the following content:

```
GNU nano 8.7.1 lab3_1.c
#include <stdio.h>
#include <unistd.h>

int main() {
    printf("I am starting...\n");
    for (int i = 1; i <= 30; i++) {
        sleep(1);
    }
    printf("I am finished.\n");
    return 0;
}
```



```
root@Ubuntu:~/biraj/2ndir# gcc lab3_1.c
root@Ubuntu:~/biraj/2ndir# gcc lab3_1.c -o lab3_1
root@Ubuntu:~/biraj/2ndir# ./lab3_1
I am starting...
█
```



```
root@Ubuntu:~/biraj/2ndir# gcc lab3_1.c
root@Ubuntu:~/biraj/2ndir# gcc lab3_1.c -o lab3_1
root@Ubuntu:~/biraj/2ndir# ./lab3_1
I am starting...
I am finished.
root@Ubuntu:~/biraj/2ndir# █
```



```
root@Ubuntu:~/biraj/2ndir# gcc lab3_1.c
root@Ubuntu:~/biraj/2ndir# gcc lab3_1.c -o lab3_1
root@Ubuntu:~/biraj/2ndir# ./lab3_1
I am starting...
I am finished.
root@Ubuntu:~/biraj/2ndir# ./lab3_1 &
[1] 7214
I am starting...
root@Ubuntu:~/biraj/2ndir# ps aux| grep lab3_1
root      7214  0.0  0.0   2768  1756 pts/1    S   15:31   0:00  ./lab3_1
root      7225  0.0  0.0   9316  2616 pts/1    S+  15:32   0:00  grep --color=auto lab3_1
root@Ubuntu:~/biraj/2ndir#
```

Figure: Task 1 C code, commands for compilation and Execution.

Summary: The source file was compiled into an executable and run in the background. Executing the process status filtering command verified that the process remained active during its sleep cycle. After approximately 30 seconds, the program displayed "I am finished." and completed execution normally.

Task no.2

```
GNU nano 8.7.1 lab3_2.c
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t my_pid = getpid();
    pid_t my_ppid = getppid();

    printf("My PID is: %d\n", my_pid);
    printf("My Parent PID is: %d\n", my_ppid);

    printf("Sleeping for 20 seconds...\n");
    sleep(20);

    return 0;
}

root@Ubuntu:~/biraj/2ndir# nano lab3_2.c
root@Ubuntu:~/biraj/2ndir# nano lab3_1.c
root@Ubuntu:~/biraj/2ndir# nano lab3_2.c
root@Ubuntu:~/biraj/2ndir# gcc lab3_2.c -o lab3_2
root@Ubuntu:~/biraj/2ndir# ./lab3_2
My PID is: 8187
My Parent PID is: 4172
Sleeping for 20 seconds...

root@Ubuntu:~/biraj/2ndir# gcc lab3_1.c
root@Ubuntu:~/biraj/2ndir# gcc lab3_1.c -o lab3_1
root@Ubuntu:~/biraj/2ndir# ./lab3_1
I am starting...
I am finished.
root@Ubuntu:~/biraj/2ndir# ./lab3_1 &
[1] 7214
I am starting...
root@Ubuntu:~/biraj/2ndir# ps aux| grep lab3_1
root      7214  0.0  0.0   2768  1756 pts/1    S   15:31   0:00  ./lab3_1
root      7225  0.0  0.0   9316  2616 pts/1    S+  15:32   0:00  grep --color=auto lab3_1
root@Ubuntu:~/biraj/2ndir# I am finished.
^C
[1]+  Done                  ./lab3_1
root@Ubuntu:~/biraj/2ndir# ls
'OS_data_types_Q&A.pdf'  a.out  data.c  data2.c  hi.c  lab3_1.c
'Week 1'                data  data2  file.txt  lab3_1  newfile1.c
root@Ubuntu:~/biraj/2ndir# nano lab3_2.c
root@Ubuntu:~/biraj/2ndir# nano lab3_1.c
root@Ubuntu:~/biraj/2ndir# nano lab3_2.c
root@Ubuntu:~/biraj/2ndir# gcc lab3_2.c -o lab3_2
root@Ubuntu:~/biraj/2ndir# ./lab3_2
My PID is: 8187
My Parent PID is: 4172
Sleeping for 20 seconds...
root@Ubuntu:~/biraj/2ndir# ./lab3_2 &
[1] 8198
My PID is: 8198
My Parent PID is: 4172
Sleeping for 20 seconds...
root@Ubuntu:~/biraj/2ndir#
```

Figure: Task 2 C code, Compilation and execution.

Summary: The program retrieved its own PID and its parent process PID. It remained active for 20 seconds to allow process status verification. The accompanying screenshot displays the runtime PID output, demonstrating how Linux assigns unique system identifiers to active processes.

Task No.3

```
GNU nano 8.7.1                                lab3_3.c
#include <stdio.h>

int main() {
    int num;

    printf("Enter a number (positive for success, negative for fail): ");
    scanf("%d", &num);

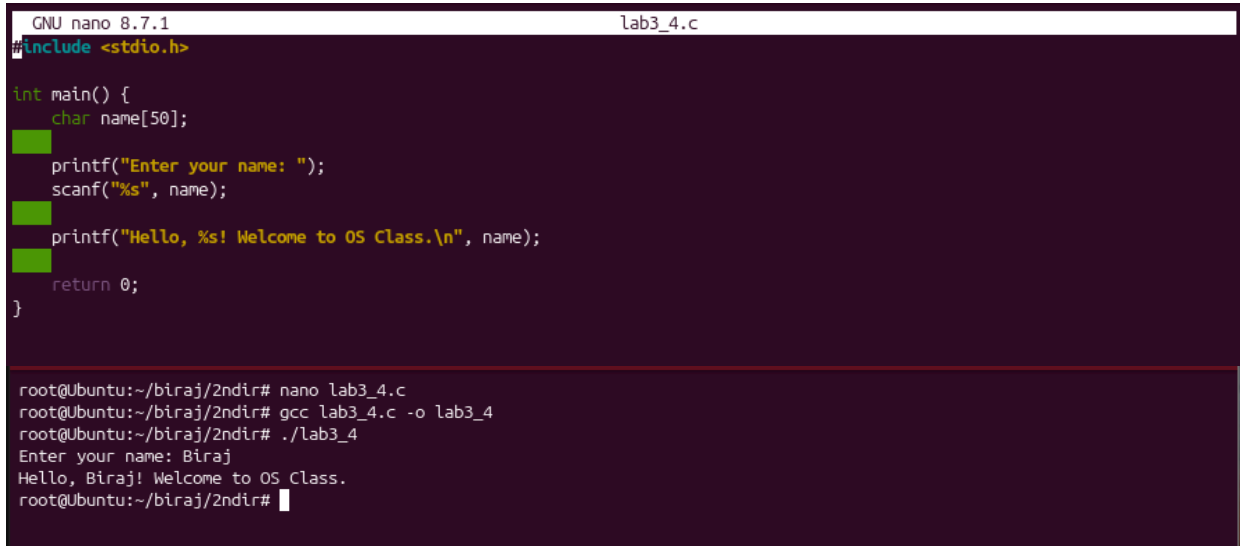
    if (num > 0) {
        printf("Success\n");
        return 0;
    } else {
        printf("Failure\n");
        return 1;
    }
}

[2]+  Done                                ./lab3_2
root@Ubuntu:~/biraj/2ndir# nano lab3_3.c
root@Ubuntu:~/biraj/2ndir# gcc lab3_3.c -o lab3_3
root@Ubuntu:~/biraj/2ndir# ./lab3_3
Enter a number (positive for success, negative for fail): 6
Success
root@Ubuntu:~/biraj/2ndir# ./lab3_3
Enter a number (positive for success, negative for fail): -2
Failure
root@Ubuntu:~/biraj/2ndir#
```

Figure: Task 3 C code, Compilation and Execution.

Summary: The program evaluated a user-provided integer using an if/else conditional block to determine its final termination state. A positive value triggered a "Success" message and returned an exit status of 0, whereas a negative value prompted a "Failure" output and returned a status of 1. These operational return states can be verified inside the shell environment by checking the exit code variable immediately following execution.

Task no.4



```
GNU nano 8.7.1 lab3_4.c
#include <stdio.h>

int main() {
    char name[50];
    printf("Enter your name: ");
    scanf("%s", name);
    printf("Hello, %s! Welcome to OS Class.\n", name);
    return 0;
}

root@Ubuntu:~/biraj/2ndir# nano lab3_4.c
root@Ubuntu:~/biraj/2ndir# gcc lab3_4.c -o lab3_4
root@Ubuntu:~/biraj/2ndir# ./lab3_4
Enter your name: Biraj
Hello, Biraj! Welcome to OS Class.
root@Ubuntu:~/biraj/2ndir#
```

Figure: Task 4 C code, Compilation and Execution.

Summary: The program captured a string from standard input and displayed a customized greeting to standard output. The accompanying screenshot confirms successful execution by displaying the user input alongside the resulting welcome message.

Task No.5

```
GNU nano 8.7.1 lab3_5.c
int main() {
    int choice;

    printf("Current PID: %d\n", getpid());
    printf("Do you want to continue? (1 for Yes, 0 for No): ");
    scanf("%d", &choice);

    if (choice == 1) {
        printf("Continuing...\n");
        sleep(5);
        return 0;
    } else {
        printf("Exiting...\n");
        return 1;
    }
}

root@Ubuntu:~/biraj/2ndir# nano lab3_5.c
root@Ubuntu:~/biraj/2ndir# gcc lab3_5.c -o lab3_5
root@Ubuntu:~/biraj/2ndir# ./lab3_5
Current PID: 8534
Do you want to continue? (1 for Yes, 0 for No): Yes
Exiting...
root@Ubuntu:~/biraj/2ndir#
```

Figure: Task 5 C code, Compilation and Execution.

Summary: The program outputted its current PID and prompted the user for a control input to dictate execution behavior. Selecting the validation option executed a timed pause before returning a successful exit code, while selecting the alternative option triggered an immediate termination with an error status. This pattern illustrates how internal conditional logic regulates both runtime execution flow and the final termination status reported to the operating system.